

BÀI TỔNG HỢP PHẦN 8

COLLECTIONS & GENERICS

Đề tài: Quản lý yêu cầu hỗ trợ sinh viên

Mục tiêu: tổng hợp Dictionary, HashSet, Queue, Stack, Generic Method, Generic Class, Generic Constraints, Interface, Validation và Exception.

Phạm vi bài: chưa dùng Delegate, Lambda, LINQ, File/JSON hay async/await. Mục tiêu là ôn chắc Phần 8 và biết phối hợp các cấu trúc dữ liệu.

I. Kiến thức được sử dụng

- Dictionary<TKey, TValue>
- HashSet<T>
- Queue<T>
- Stack<T>
- Generic Method
- Generic Class
- Generic Constraints
- Interface
- Encapsulation
- Validation
- Exception

Chưa sử dụng: Delegate, Lambda, LINQ, File và JSON, async/await.

II. Cấu trúc dự án đề xuất

```
Interfaces/  
    IEntity.cs  
    IDisplayable.cs  
  
Enums/  
    SupportCategory.cs  
    RequestStatus.cs  
  
Models/  
    SupportRequest.cs  
  
Repositories/  
    EntityRepository.cs  
  
Services/  
    SupportRequestManager.cs  
  
Utilities/  
    GenericHelper.cs  
  
Program.cs
```

Bạn có thể xem cách chia thư mục này như một quy ước tổ chức code. Ở giai đoạn hiện tại chưa cần hiểu sâu kiến trúc Repository/Service; chỉ cần biết file nào có trách nhiệm gì.

III. Phần 1 - Tạo các interface

1. Interface IEntity

```
public interface IEntity  
{  
    string Id { get; }  
}
```

Vai trò: bảo đảm mọi entity được lưu trong repository đều có Id. Nhờ constraint, Generic Repository có thể truy cập item.Id.

2. Interface IDisplayable

```
public interface IDisplayable
{
    void DisplayInfo();
}
```

Vai trò: bảo đảm object có khả năng tự hiển thị thông tin. Nhờ constraint, Generic Repository có thể gọi item.DisplayInfo().

IV. Phần 2 - Tạo Enum

1. SupportCategory

```
public enum SupportCategory
{
    Academic,
    Tuition,
    Technology,
    Account,
    Other
}
```

Giá trị	Ý nghĩa
Academic	Học tập, đăng ký môn
Tuition	Học phí
Technology	Lỗi hệ thống, thiết bị
Account	Tài khoản sinh viên
Other	Vấn đề khác

2. RequestStatus

```
public enum RequestStatus
{
    Pending,
    Completed
}
```

Pending: đang chờ xử lý. Completed: đã xử lý.

V. Phần 3 - Class SupportRequest

```
public class SupportRequest : IEntity, IDisplayable
```

1. Property bắt buộc

```
public string Id { get; }
public string StudentName { get; }
public string Title { get; }
public SupportCategory Category { get; }
public DateTime CreatedAt { get; }
public RequestStatus Status { get; private set; }
```

2. Constructor

```
public SupportRequest(  
    string id,  
    string studentName,  
    string title,  
    SupportCategory category)
```

- Kiểm tra id.
- Kiểm tra studentName.
- Kiểm tra title.
- Dùng Trim() trước khi gán.
- Gán Category = category.
- Tự gán CreatedAt = DateTime.Now.
- Đặt Status = RequestStatus.Pending.

3. Validation

Nếu id, studentName hoặc title là null, chuỗi rỗng hoặc chỉ có khoảng trắng thì ném ArgumentException.

```
if (string.IsNullOrEmpty(id))  
{  
    throw new ArgumentException(  
        "Id cannot be null or whitespace.",  
        nameof(id));  
}
```

4. Method thay đổi trạng thái

```
public void MarkAsCompleted()
```

- Nếu Status đã là Completed thì ném InvalidOperationException.
- Nếu chưa, đổi Status thành RequestStatus.Completed.

```
public void MarkAsPending()
```

Dùng cho chức năng hoàn tác: đổi trạng thái trở lại RequestStatus.Pending.

5. Hiển thị

```
public void DisplayInfo()
```

Ví dụ kết quả:

```
Id: YC01  
Student: Hung  
Title: Cannot log in  
Category: Account  
Created at: 02/09/2026 10:20  
Status: Pending
```

Nên override thêm ToString() để Console.WriteLine(request) hiển thị có ý nghĩa.

VI. Phần 4 - Generic Repository

```
public class EntityRepository<T>  
    where T : class, IEntity, IDisplayable
```

Constraint	Ý nghĩa
class	T phải là reference type
IEntity	T phải có Id
IDisplayable	T phải có DisplayInfo()

Collection bên trong

```
private readonly Dictionary<string, T> _items;
public EntityRepository()
{
    _items = new Dictionary<string, T>(
        StringComparer.OrdinalIgnoreCase);
}
```

Nhờ StringComparer.OrdinalIgnoreCase, YC01, yc01 và Yc01 được xem là cùng một key.

Method 1 - Add

```
public void Add(T item)
```

- Kiểm tra item == null.
- Không cho phép trùng Id.
- Nếu trùng ném InvalidOperationException.
- Nếu hợp lệ, thêm bằng _items.Add(item.Id, item).

Method 2 - FindById

```
public T? FindById(string id)
```

- Validation id.
- Dùng Trim().
- Tìm thấy trả về entity.
- Không tìm thấy trả về null.
- Có thể dùng _items.TryGetValue(...).

Method 3 - ContainsId

```
public bool ContainsId(string id)
```

- Có thể tái sử dụng FindById() hoặc dùng ContainsKey().

Method 4 - RemoveById

```
public bool RemoveById(string id)
```

- Validation thông qua FindById() hoặc method riêng.
- Tìm thấy thì xóa và trả true.
- Không tìm thấy trả false.

Method 5 - GetCount

```
public int GetCount()
```

- Trả về số entity trong repository.

Method 6 - GetAllItems

```
public List<T> GetAllItems()
```

- Không trả trực tiếp collection nội bộ.
- Tạo danh sách mới từ _items.Values.

Method 7 - DisplayAll

```
public void DisplayAll()
```

- Nếu repository rỗng, hiển thị thông báo rồi return.
- Duyệt entity.
- Gọi item.DisplayInfo() để chứng minh interface constraint hoạt động.

VII. Phần 5 - Class SupportRequestManager

```
public class SupportRequestManager
```

Class này phối hợp các collection để xử lý nghiệp vụ. Ở giai đoạn hiện tại chỉ cần hiểu luồng dữ liệu, chưa cần học sâu kiến trúc Service/Repository.

Các field bắt buộc

```
private readonly EntityRepository<SupportRequest> _repository;
private readonly Queue<SupportRequest> _pendingQueue;
private readonly Stack<SupportRequest> _processedHistory;
private readonly HashSet<string> _queuedRequestIds;

public SupportRequestManager()
{
    _repository = new EntityRepository<SupportRequest>();
    _pendingQueue = new Queue<SupportRequest>();
    _processedHistory = new Stack<SupportRequest>();
    _queuedRequestIds = new HashSet<string>(
        StringComparer.OrdinalIgnoreCase);
}
```

VIII. Vai trò của từng collection

Collection	Vai trò
Dictionary<string, SupportRequest>	Lưu toàn bộ yêu cầu, tìm theo Id, chống trùng key.
Queue<SupportRequest>	Hàng chờ xử lý theo FIFO: vào trước xử lý trước.
Stack<SupportRequest>	Lịch sử xử lý theo LIFO: xử lý gần nhất hoàn tác trước.
HashSet<string>	Lưu Id đang nằm trong Queue để kiểm tra trùng nhanh.

IX. Các method của SupportRequestManager

Method 1 - AddRequest

```
public void AddRequest(SupportRequest request)
```

- Kiểm tra request == null.
- Gọi _repository.Add(request).
- Chưa tự động đưa vào Queue.

Method 2 - EnqueueRequest

```
public void EnqueueRequest(string id)
```

- Tìm yêu cầu theo Id trong repository.
- Nếu không tìm thấy, ném InvalidOperationException.
- Nếu Id đã có trong _queuedRequestIds, ném InvalidOperationException.
- Nếu request đã Completed, không cho thêm lại (trừ khi đã Undo về Pending).
- Nếu hợp lệ: _pendingQueue.Enqueue(request) và _queuedRequestIds.Add(request.Id).

Method 3 - ProcessNextRequest

```
public SupportRequest ProcessNextRequest()
```

- Nếu Queue rỗng, ném InvalidOperationException.
- Lấy phần tử đầu bằng Dequeue().
- Xóa Id khỏi HashSet.
- Gọi request.MarkAsCompleted().
- Push request vào Stack lịch sử.
- Trả về request vừa xử lý.

Method 4 - UndoLastProcessedRequest

```
public SupportRequest UndoLastProcessedRequest()
```

- Nếu Stack rỗng, ném InvalidOperationException.
- Lấy request gần nhất bằng Pop().
- Gọi request.MarkAsPending().
- Đưa request trở lại Queue.
- Thêm lại Id vào HashSet.
- Trả về request vừa hoàn tác.

Method 5 - FindRequestById

```
public SupportRequest? FindRequestById(string id)
```

- Tái sử dụng _repository.FindById(id).

Method 6 - ContainsRequest

```
public bool ContainsRequest(string id)
```

- Tái sử dụng repository.

Method 7 - GetTotalRequestCount

```
public int GetTotalRequestCount()
```

- Trả về tổng số request đang được repository quản lý.

Method 8 - GetPendingRequestCount

```
public int GetPendingRequestCount()
```

- Trả về `_pendingQueue.Count`.

Method 9 - GetProcessedHistoryCount

```
public int GetProcessedHistoryCount()
```

- Trả về `_processedHistory.Count`.
- Đây là số phần tử hiện đang trong Stack lịch sử, không nhất thiết là tổng số từng xử lý nếu đã Undo.

Method 10 - DisplayAllRequests

```
public void DisplayAllRequests()
```

- Tái sử dụng `_repository.DisplayAll()`.

Method 11 - DisplayPendingQueue

```
public void DisplayPendingQueue()
```

- Nếu Queue rỗng, hiển thị thông báo.
- Không dùng `Dequeue()` chỉ để hiển thị.
- Dùng `foreach` để không làm mất dữ liệu.

Method 12 - DisplayProcessedHistory

```
public void DisplayProcessedHistory()
```

- Nếu Stack rỗng, hiển thị thông báo.
- Không dùng `Pop()` chỉ để hiển thị.
- Dùng `foreach`; phần tử trên cùng sẽ được hiển thị trước.

X. Phần 6 - Generic Method

```
public static class GenericHelper
{
    public static void DisplayValue<T>(
        string label,
        T value)
    {
        Console.WriteLine(
            $"{label}: {value}");
    }
}
```

Đây là Generic Method vì `<T>` nằm sau tên method. Có thể dùng với `int`, `string`, `SupportRequest`, `RequestStatus`...

XI. Yêu cầu trong Program.cs

Giai đoạn 1 - Khởi tạo

SupportRequestManager manager = new SupportRequestManager();

Giai đoạn 2 - Tạo ít nhất bốn yêu cầu

```
YC01 - Hung - Cannot log in - Account  
YC02 - Lam - Tuition information - Tuition  
YC03 - An - Course registration error - Academic  
YC04 - Duy - Website loading error - Technology
```

Giai đoạn 3 - Hiển thị toàn bộ

Gọi manager.DisplayAllRequests(). Kiểm tra đủ 4 request và Status ban đầu đều là Pending.

Giai đoạn 4 - Tìm kiếm

Tìm manager.FindRequestById("yc02") trong khi mã thật là YC02. Kết quả vẫn phải tìm thấy.

Giai đoạn 5 - Đưa vào Queue

Enqueue theo thứ tự YC01, YC02, YC03 rồi DisplayPendingQueue(). Thứ tự hiển thị phải giữ YC01 -> YC02 -> YC03.

Giai đoạn 6 - Xử lý yêu cầu

Gọi ProcessNextRequest() hai lần. Lần 1 phải là YC01, lần 2 là YC02. Sau đó Queue còn YC03; Stack có YC01 và YC02; hai request đã xử lý có Status = Completed.

Giai đoạn 7 - Hiển thị lịch sử

DisplayProcessedHistory() dự kiến hiển thị YC02 trước YC01 do Stack là LIFO.

Giai đoạn 8 - Hoàn tác

UndoLastProcessedRequest() phải hoàn tác YC02. YC02 về Pending và được đưa lại cuối Queue. Stack chỉ còn YC01.

Giai đoạn 9 - Hiển thị số lượng

Dùng GenericHelper.DisplayValue() để in tổng request, số pending và số phần tử trong processed history.

XII. Kiểm thử exception

Mỗi trường hợp lỗi phải có một try-catch riêng. Nếu đặt nhiều lỗi trong cùng một try, lỗi đầu tiên sẽ làm các test phía sau không chạy.

1. Tạo request có Id rỗng

Tạo SupportRequest với id = " ". Bắt ArgumentException.

2. Thêm request trùng Id

Tạo request mới với id = "yc01" khi YC01 đã tồn tại. Bắt `InvalidOperationException`. Phải xem YC01 và yc01 là trùng.

3. Đưa cùng request vào Queue hai lần

Gọi `EnqueueRequest()` hai lần với cùng Id (khác hoa thường cũng phải bị phát hiện). Bắt `InvalidOperationException`.

4. Enqueue Id không tồn tại

Gọi `EnqueueRequest("YC99")`. Bắt `InvalidOperationException`.

5. Process khi Queue rỗng

Tạo `SupportRequestManager` mới rồi gọi `ProcessNextRequest()`. Bắt `InvalidOperationException`.

6. Undo khi Stack rỗng

Tạo `SupportRequestManager` mới rồi gọi `UndoLastProcessedRequest()`. Bắt `InvalidOperationException`.

XIII. Quy tắc quan trọng

Không sửa collection trong foreach

Không `Dequeue/Pop/Remove` collection đang được duyệt chỉ để hiển thị.

Không trả trực tiếp collection nội bộ

Nếu cần trả dữ liệu, trả bản sao hoặc danh sách mới thay vì trả field collection.

Mỗi lỗi một try-catch

Tách test lỗi để bảo đảm tất cả trường hợp đều được chạy.

Không dùng LINQ

Bài này giải quyết bằng foreach và các collection đã học: Dictionary, Queue, Stack, HashSet.

XIV. Câu hỏi phải tự trả lời sau khi làm

1. Vì sao dùng Dictionary để quản lý toàn bộ yêu cầu?
2. Vì sao dùng Queue cho yêu cầu chờ xử lý?
3. Vì sao dùng Stack cho chức năng hoàn tác?
4. Vì sao cần thêm HashSet trong khi đã có Dictionary?
5. where T : class bảo đảm điều gì?
6. where T : IEntity cho phép dùng thành viên nào?
7. where T : IDisplayable cho phép gọi method nào?
8. Vì sao Add() trong Generic Class không cần viết Add<T>()?
9. Vì sao DisplayValue<T>() lại cần khai báo <T>?
10. Vì sao mỗi test exception cần một try-catch riêng?

XV. Tiêu chí chấm bài

Nội dung	Điểm
SupportRequest và validation	10
IEntity, IDisplayable	5
Generic Repository và constraints	15
Sử dụng Dictionary	10
Sử dụng Queue đúng FIFO	10
Sử dụng Stack đúng LIFO	10
Sử dụng HashSet chống trùng Queue	10
Generic Method	5
Xử lý trạng thái yêu cầu	10
Exception và kiểm thử	10
Naming, encapsulation, trình bày	5
Tổng	100

XVI. Thứ tự thực hiện đề xuất

- Bước 1: Interface và enum
- Bước 2: SupportRequest
- Bước 3: EntityRepository<T>
- Bước 4: Kiểm tra repository riêng
- Bước 5: SupportRequestManager
- Bước 6: Queue và HashSet
- Bước 7: Stack và Undo
- Bước 8: GenericHelper
- Bước 9: Program
- Bước 10: Các test exception

Cách học đề xuất: xem lại tài liệu cũ khi quên cú pháp là hoàn toàn bình thường. Hãy tự quyết định collection nào giải quyết yêu cầu nào; chỉ tra cú pháp/phương thức khi cần.